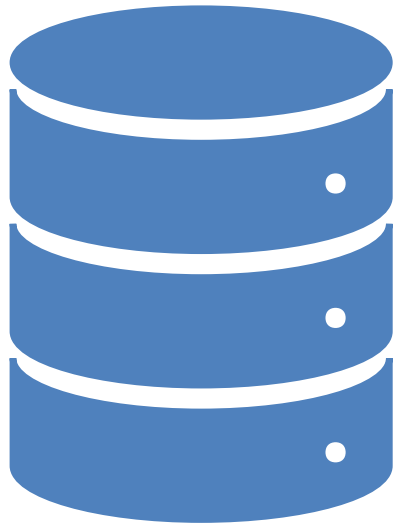




Introduction to MySQL

Tables and Database in PHP

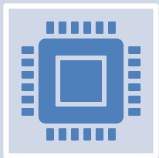
What is MySQL?



MySQL is an **open-source Relational Database Management System (RDBMS)** that is widely used with PHP to build dynamic web applications.



It organizes data in **tables** and allows **Structured Query Language (SQL)** operations to retrieve and manipulate data efficiently.



PHP + MySQL is a classic combo used in many CMS platforms like WordPress, Joomla, Drupal, etc.

Features of MySQL

- **Open Source** – Freely available and customizable.
- **Fast & Reliable** – Optimized for speed and performance.
- **Cross-Platform** – Works on Windows, Linux, macOS.
- **Scalable** – Suitable for small to large applications.
- **Secure** – Offers user management and SSL support.
- **Support for Large Databases** – Handles up to 50 million rows or more.
- **Multi-user Access** – Allows multiple users to access the database concurrently.

Why Use PHP with MySQL?

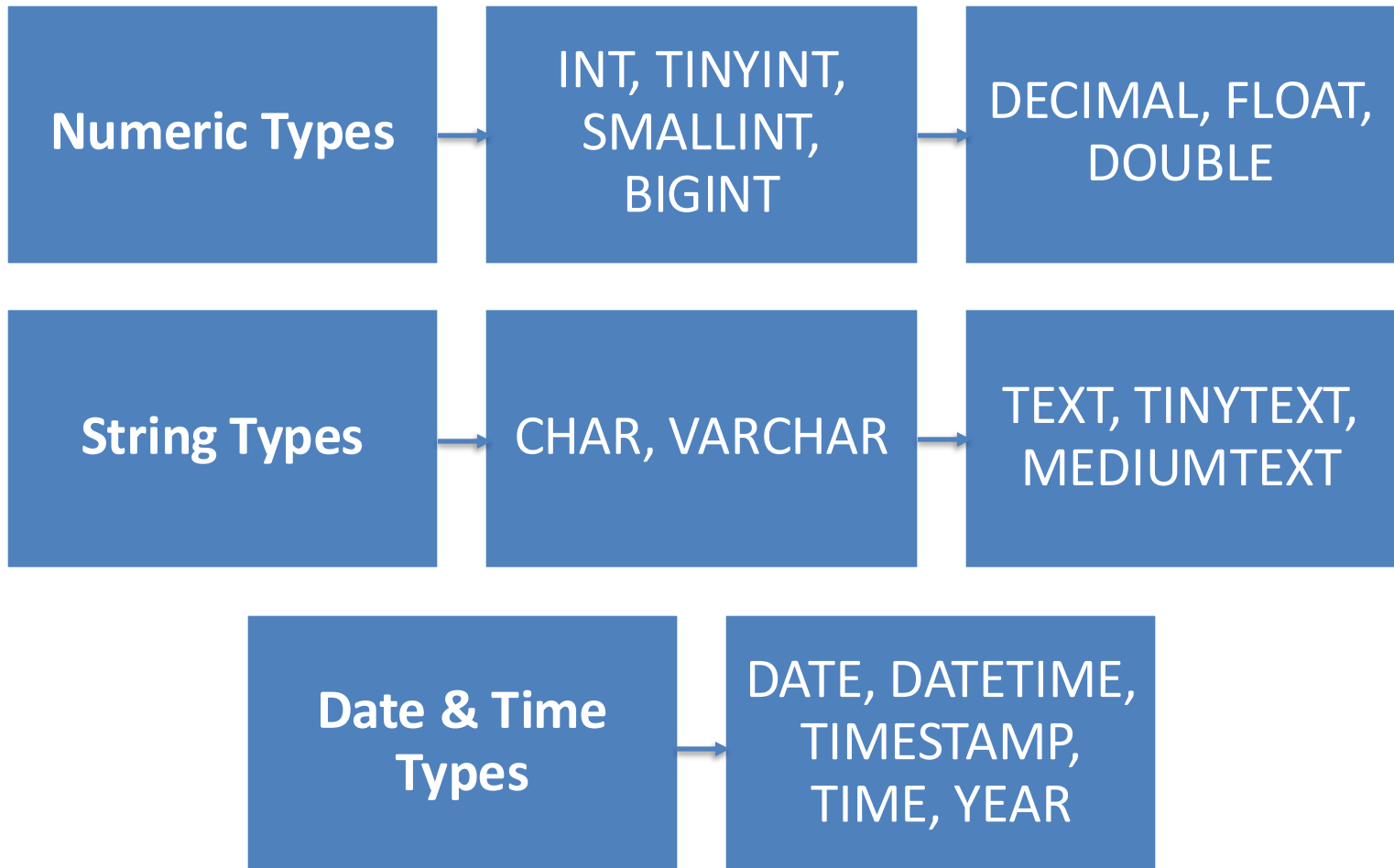
- PHP can connect to MySQL databases.
- Perform operations: insert, update, delete, retrieve data using SQL.
- Used in CMS like WordPress, Joomla, Drupal.



Basic Concepts

Term	Description
Database	Collection of structured data stored electronically
Table	Collection of related data in rows and columns
Row (Record)	Single, complete set of data in a table
Column (Field)	Specific attribute of data (like name, age, etc.)
Primary Key	Uniquely identifies each record in a table
Foreign Key	Links one table with another
Query	Instruction to interact with the database

Data Types in MySQL



SQL Commands in MySQL

A. Data Definition Language (DDL)

Command	Purpose
CREATE	Creates database or table
ALTER	Modifies existing table
DROP	Deletes table or database
TRUNCATE	Deletes all rows from a table

SQL Commands in MySQL

B. Data Manipulation Language (DML)

Command	Purpose
INSERT	Adds data
UPDATE	Modifies data
DELETE	Removes data

C. Data Query Language (DQL)

Command	Purpose
SELECT	Retrieves data from table

D. Data Control Language (DCL)

Command	Purpose
GRANT	Gives access privileges
REVOKE	Removes privileges

Database

A **database** is a container that holds data in a structured form, using tables.

- `CREATE DATABASE mydatabase;`
`//Creating database`
- `USE mydatabase; //using database`

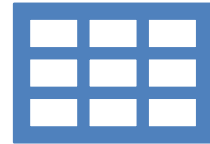


Tables

A **table** is a collection of related data entries and consists of **columns (fields)** and **rows (records)**.

Example table:

```
CREATE TABLE students ( id INT  
    AUTO_INCREMENT PRIMARY KEY, name  
    VARCHAR(50), email VARCHAR(100), age INT  
);
```



Sample SQL Table

```
CREATE TABLE students (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100),  
  email VARCHAR(100),  
  course VARCHAR(50)  
);
```

PHP + MySQL: Connecting to Database

```
<?php
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "school";

// Create connection
$conn = new mysqli($servername, $username,
$password, $dbname);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn-
>connect_error);
}
echo "Connected successfully!";
?>
```

Inserting Data Example

```
$sql = "INSERT INTO students (name, email, course)  
VALUES ('ABC', 'abc@example.com', 'Computer Science')";
```

```
if ($conn->query($sql) === TRUE) {  
    echo "New record created successfully!";  
} else {  
    echo "Error: " . $sql . "<br>" . $conn->error;  
}
```

Fetching Data Example

```
$sql = "SELECT id, name, email FROM students";  
$result = $conn->query($sql);  
  
if ($result->num_rows > 0) {  
    while($row = $result->fetch_assoc()) {  
        echo "ID: " . $row["id"]. " | Name: " . $row["name"]. " | Email: " .  
        $row["email"]. "<br>";  
    }  
} else {  
    echo "0 results";  
}
```

Keys in MySQL

Primary Key

- Uniquely identifies each record.
- Cannot be NULL.

Example:

```
id INT PRIMARY KEY
```



Keys in MySQL

Foreign Key

- Links two tables together.
- Enforces referential integrity.

Example:

```
FOREIGN KEY (student_id)  
REFERENCES students(id)
```



MySQL Constraints

Constraint	Description
NOT NULL	Field must have a value
UNIQUE	All values in column must be different
PRIMARY KEY	Unique + Not Null
FOREIGN KEY	Maintains referential integrity
DEFAULT	Assigns default value
CHECK	Ensures data meets condition

MySQL Functions

Category	Example Functions
String	CONCAT(), LENGTH(), UPPER()
Numeric	ROUND(), FLOOR(), ABS()
Date/Time	NOW(), CURDATE(), DATEDIFF()
Aggregate	COUNT(), SUM(), AVG(), MAX(), MIN()

MySQL Joins

Used to combine rows from two or more tables:

Type	Description
INNER JOIN	Matches records in both tables
LEFT JOIN	All records from left + matched from right
RIGHT JOIN	All records from right + matched from left
FULL JOIN	All records when there is a match in one of the tables (Not supported directly in MySQL, but can be emulated)

Example

```
SELECT students.name, marks.score  
FROM students  
INNER JOIN marks ON students.id = marks.student_id;
```

MySQL User and Privileges

Create User

```
CREATE USER 'username'@'localhost' IDENTIFIED BY 'password';
```

Grant Privileges

```
GRANT ALL PRIVILEGES ON database_name.* TO 'username'@'localhost';
```

Advantages of MySQL

Open-source and free.

Easy integration with programming languages.

Scalable and fast performance.

Good support and large community.

Cross-platform support.

Limitations of MySQL

Lacks some advanced features of other RDBMS like Oracle or PostgreSQL.

A large blue downward-pointing arrow connecting the first limitation to the second.

Poor performance with large JOINS.

A large blue downward-pointing arrow connecting the second limitation to the third.

No full support for transactional databases (in older versions).

include and require in PHP

- PHP allows code reuse and modularity through include and require.
- These are used to insert the content of one PHP file into another.
- The 'include' (or require) statement copies all of the text, code, and mark-up from the defined file into the include statement's target file.
- When you want to use the same PHP, HTML, or text on different pages of a website, including files comes in handy.



Why Use include or require?



Promotes **code reusability**.



Helps organize code into **multiple files**.



Commonly used for:

- Headers and footers
- Navigation menus
- Database connections
- Configuration files

1. include

- **Purpose:** Includes and evaluates the specified file.
- **If file is missing:** PHP **continues** execution with a **warning**.

Syntax: include 'filename.php';

```
// header.php
```

```
echo "<h1>Welcome to My Website</h1>";
```

```
// index.php
```

```
include 'header.php';
```

```
echo "Home Page Content";
```

2. require

- **Purpose:** Works the same as include, but with stricter error handling.
- **If file is missing:** PHP **stops execution** and throws a **fatal error**.

Syntax: `require 'filename.php';`

```
require 'config.php';  
echo "Database connected!";
```

Comparison b/w include & require

Feature	include	require
Error on missing file	Warning (script continues)	Fatal Error (script stops)
File loaded	Yes	Yes
Use case	Optional files	Critical files (e.g., DB connection)

3. include_once and require_once

- These prevent multiple inclusions of the **same file**, which can cause errors like "**function already declared**".

Syntax:

```
include_once 'filename.php';
```

```
require_once 'filename.php';
```

```
// db.php
```

```
function connectDB() { /* ... */ }
```

```
// index.php
```

```
include_once 'db.php';
```

```
include_once 'db.php'; // Will not include again
```